



河南大學
HENAN UNIVERSITY

第2章 运算方法和运算器

2.1 数据与文字的表示方法 (掌握)

2.2 定点加法、减法运算 (掌握)

2.3 定点乘法运算 (理解)

2.4 定点除法运算 (理解)

2.5 定点运算器的组成 (了解)

2.6 浮点运算方法和浮点运算器 (理解)

>>> 2.1 数据与文字的表达方法

2.1.1 数据格式

2.1.2 数的机器码表示

2.1.3 字符与字符串的表示方法

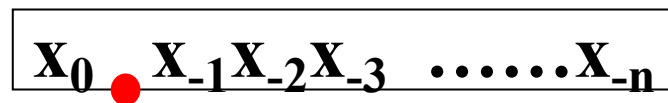
2.1.4 汉字的表示方法

2.1.5 校验码

>>> 2.1.1 数据格式——定点数（两种）

➤ **定点数：** 小数点固定在某一位置的数据； 设采用 $n+1$ 位数据

□ **纯小数：**



✓ **表示形式**

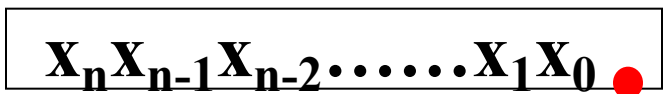
有符号数

$$x = x_s x_{-1} x_{-2} \dots x_{-n} \quad |x| \leq 1 - 2^{-n} \quad ; x_s \text{ 为符号位}$$

无符号数

$$x = x_0 x_{-1} x_{-2} \dots x_{-n} \quad 0 \leq x \leq 1 - 2^{-n} \quad ; x_0 = 0$$

□ **纯整数：**



✓ **表示形式**

有符号数

$$x = x_s x_{n-1} \dots x_1 x_0 \quad |x| \leq 2^n - 1 \quad ; x_s \text{ 为符号位}$$

无符号数

$$x = x_n x_{n-1} \dots x_1 x_0 \quad 0 \leq x \leq 2^{n+1} - 1 \quad ; x_n \text{ 为数值位}$$

注意

小数点位置是
机器约定好的，
一般不会保存。

>>> 定点数原码表示时最值

➤ 定点数：最值情况

设采用n+1位数据

□ 纯小数：（原码表示时最值情况）

✓ 表示形式 $2^0 \quad 2^{-1} \quad 2^{-2} \quad 2^{-3} \quad \dots \quad 2^{-(n-1)} \quad 2^{-n}$

✓ 最大正数

0	1	1	1	...	...	1	1
---	---	---	---	-----	-----	---	---

✓ 最大正数值为： $1-2^{-n}$,

✓ 最小正数

0	0	0	0	...	...	0	1
---	---	---	---	-----	-----	---	---

✓ 最小正数的值为： 2^{-n}

>>> 定点数原码表示时最值

➤ 定点数：最值情况

设采用n+1位数据

□ 纯小数：（原码表示时最值情况）

✓ 表示形式

✓ 绝对值最大负数

$$\begin{array}{ccccccc} 2^0 & 2^{-1} & 2^{-2} & 2^{-3} & & 2^{-(n-1)} & 2^{-n} \\ \hline 1 & 1 & 1 & 1 & \dots & 1 & 1 \end{array}$$

✓ 绝对值最大负数值为：- (1-2⁻ⁿ)

>>> 定点数补码表示时最值

➤ 定点数：最值情况

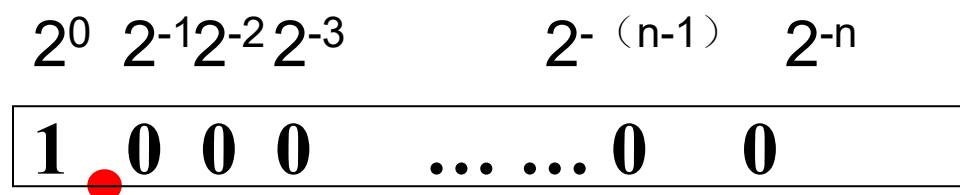
□ 纯小数：（补码表示时最值情况）

✓ 表示形式

✓ 绝对值最大负数

✓ 绝对值最大负数值为：-1

- 通过2.1节公式可推出此结论



□ 推导理解（在 $n+1$ 位情况下，定点小数用原码无法表示-1.0），见下页

>>> 定点数补码表示时最值

纯小数 绝对值最大的负数值 -1，推导理解（8位为例）

补码最小值为-1.0的得出逻辑：

先看倒数第二小的值是- $(1/2 + 1/4 + 1/8 + 1/16 + 1/32 + 1/64 + 1/128) = -0.9921875$

该值的原码：1.1111111

该值的反码：1.0000000

该值的补码：1.0000001

$$\begin{array}{r} \text{再看 } -1.0 \quad - (1/2 + 1/4 + 1/8 + 1/16 + 1/32 + 1/64 + 1/128) \\ \quad \quad \quad + \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad - (1/128) \\ \hline -1.0 \end{array}$$

所以，其实-1.0的二进制原码要占九位：11.0000000 依然硬气地假设最左位为符号位。

继续，所以-1.0的二进制反码也占九位：10.1111111

末尾加1得 -1.0的二进制补码也占九位：11.0000000

但是机器是8位机，所以数值位的最高位的1就隐去不要了，得到二进制 $1.0000000 = -1.0$ (十进制)

当机器中出现一个二进制补码1.0000000时，按照计算补码的逆规则可以得到其原码值：

1. 数值位在末尾减1，也就是减 $1/128$ 。一看是7个0，这时必须从高位借位1，这个1就是数值1.0。

减去 $1/128$ 后得到 1.1111111

2. 各数值位取反得到1.0000000。这个值看上去是-0.0，但是逆计算过程中借位时引入了1.0值，所以这个数是-1.0。



➤ 定点数：最值情况

□ 纯小数：（补码表示时最值情况）

✓ 绝对值最大负数

2^0	2^{-1}	2^{-2}	2^{-3}			$2^{-(n-1)}$	2^{-n}
1	0	0	0	...	...	0	0

✓ 绝对值最大负数值为：-1.0

- 通过2.2节公式可推出此结论（定点小数中，原码无法表示-1）

□ 纯小数补码最值-1.0其他推导方法

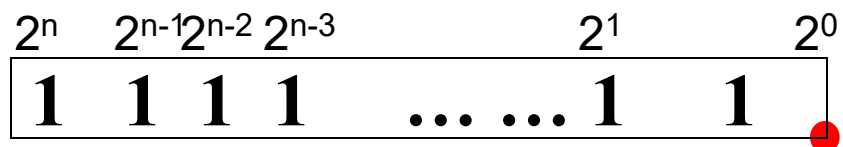
- ✓ 在 $n+1$ 位纯小数原码的表示方式中不存在-1.0，除非 $n+2$ 位能表示出：例如右图，所以 $n+1$ 位无法用原码推导出对应补码形式
- ✓ 但由于补码0的存在是唯一的，
在指定的 $n+1$ 位数的情况下，多表达了-1.0这个数



➤ 定点数：最值情况

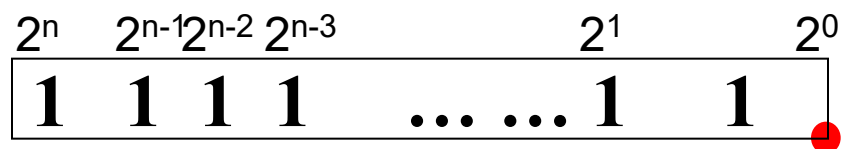
□ 纯整数：

✓ 原码表示的绝对值最大负数



- 原码表示的绝对值最大负数值为：- $(2^n - 1)$

✓ 补码表示的绝对值最大的负数



- 补码表示的绝对值最大的负数值为：- 2^n

>>> 2.1.2 数的机器码表示 (原码、补码、移码)

➤ **原码**：符号位 (0-正数, 1-负数) , 数值位与真值相同;

□ **n+1位数据的表示范围**

8位原码定点整数表示范围

$$-127 \leq X \leq +127$$

✓ 定点小数: $[-(1-2^{-n}), +(1-2^{-n})]$; 定点整数: $[-(2^n-1), +(2^n-1)]$

□ 0有两种表示法: $[+0]_{\text{原}} = 0000\ 0000$; $[-0]_{\text{原}} = 1000\ 0000$

□ 参与运算复杂, 需要将数值位与符号位分开考虑。

➤ **补码**：符号位 (0-正数, 1-负数) , 正数数值与真值相同, 负数数值各位取反末位加1;

□ **n+1位数据的表示范围**

✓ 定点小数: $[-1, +(1-2^{-n})]$; 定点整数: $[-2^n, +(2^n-1)]$

□ 0只有唯一的一种编码: $[+0]_{\text{补}} = [-0]_{\text{补}} = 0000\ 0000$

8位补码定点整数表示范围

$$-128 \leq X \leq +127$$

□ 只要结果不溢出, 可将补码符号位与数值位一起参与运算。

□ $[[x]_{\text{补}}]_{\text{补}} = [x]_{\text{原}}$

>>> 由原码求补码

➤ 由原码求补码的简便原则(负数)

方法一：

除符号位以外,其余各位按位取反, 末位加1;

方法二：

除符号位以外,从最低位开始,遇到的第一个1以前的各位保持不变, 之后各位取反。

例: $[X]_{\text{原}} = 1\ 1\ 0\ 1\ 1\ 0\ \mathbf{1}\ 0\ 0$

$[X]_{\text{补}} = 1\ 0\ 1\ 0\ 0\ 1\ \mathbf{1}\ 0\ 0$

>>> 求相反数的补码

➤ 由 $[X]_{\text{补}}$ 求 $[-X]_{\text{补}}$

□ 连同符号位的所有位一起取反，末位加1；

➤ 例： $X = -0.010\ 1011$

□ $[X]_{\text{补}} = 1.101\ 0101$; $[-X]_{\text{补}} = 0.010\ 1011$;

➤ 即： $[X]_{\text{补}} = 1\ 1\ 0\ 1\ 0\ 1\ 0\ 1$

$0\ 0\ 1\ 0\ 1\ 0\ 1\ 0$

1

+

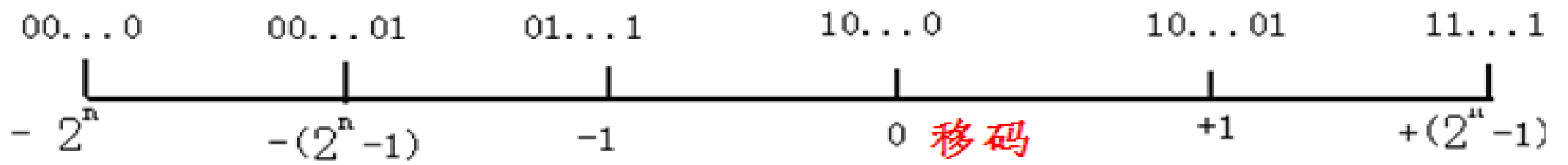
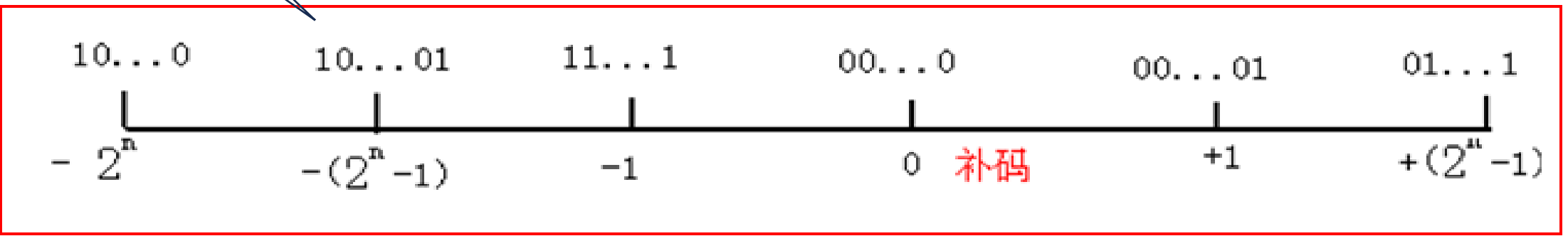
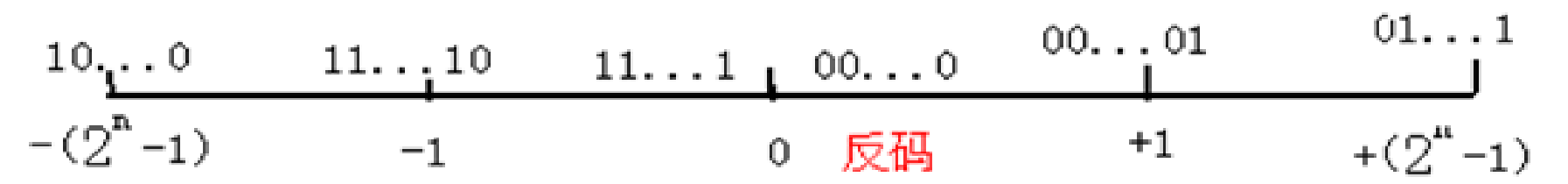
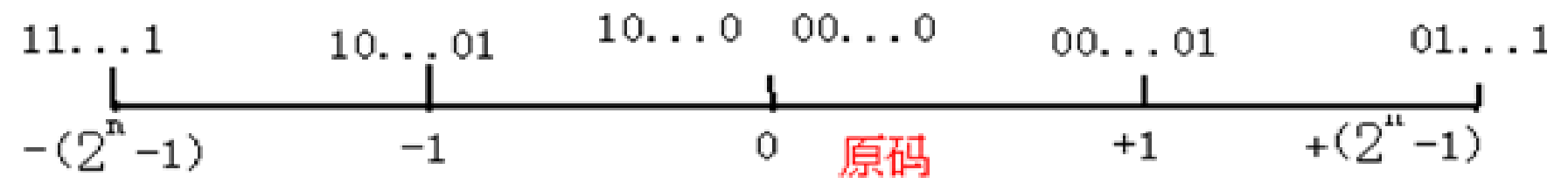
$[-X]_{\text{补}} = 0\ 0\ 1\ 0\ 1\ 0\ 1\ 1$

由 $[-X]_{\text{补}}$ 求 $[X]_{\text{补}}$ ，
此规则同样适用。

课本P22例6

以定点整数为例,用数轴形式说明原码、反码、补码、移码表示范围和可能的数码组合情况。

你能发现什么规律



2015年考研真题第13题

13. 由3个“1”和5个“0”组成的8位二进制补码，能表示的最小整数是 ____。

☐ A -126

☒ B -125

☐ C -32

☐ D -3

提交

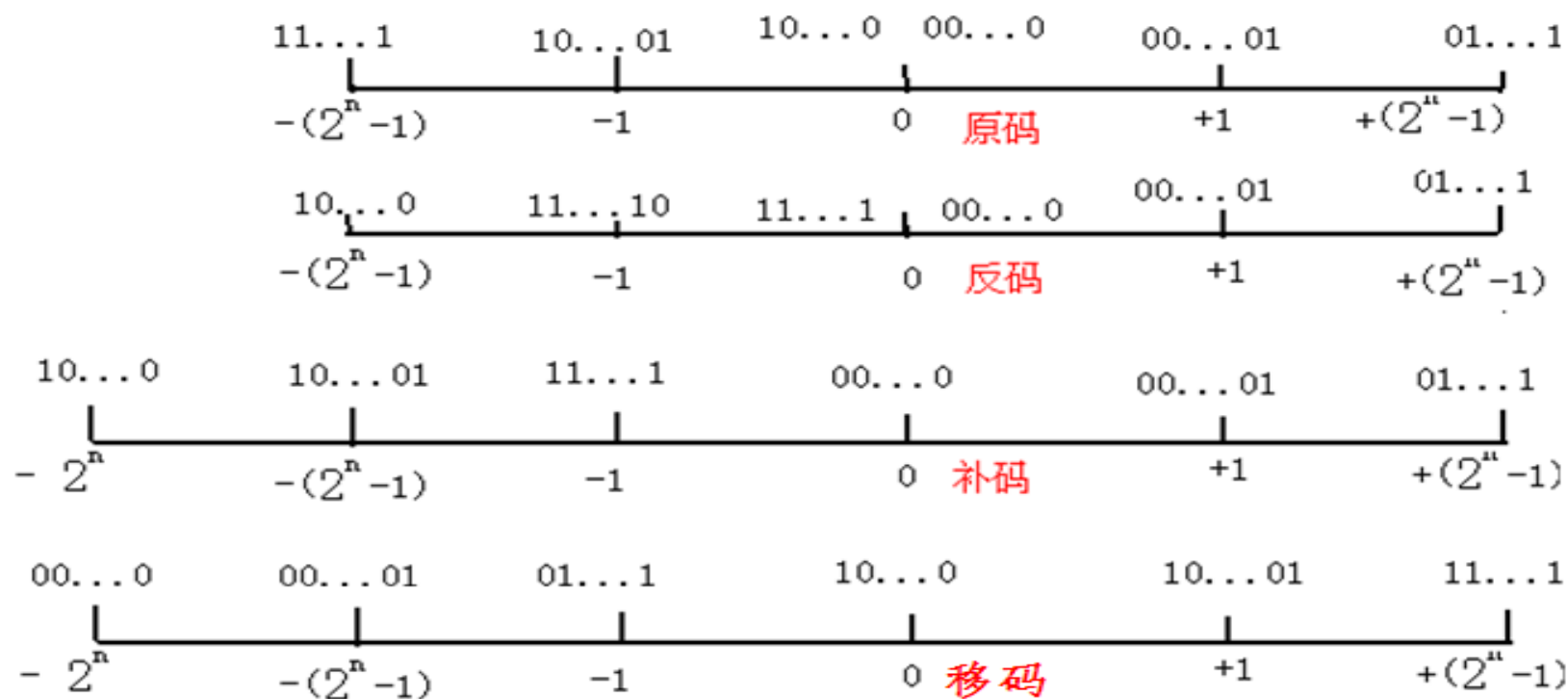
>>> 解答

➤ 3个“1”和5个“0”，符号位为“1”，然后将其他的“1”尽量放在低位，为了得到最小的整数，这样可以使负数的绝对值最大，从而得到最小的整数。

➤ 补码：10000011

原码：11111101

真值：-125



2016考研真题第13题

有如下C语言程序段

```
Short si=-32767;
```

```
Unsigned short usi=si;
```

执行上述两条语句后，usi的值为（ ）

short为16位有符号数；

unsigned short 为16位无符号数。

- ☐ A -32767
 ☐ B 32767
 ☐ C 32768
 ☒ D 32769

提交

➤ 有如下C语言程序段

```
Short si=-32767;
```

```
Unsigned short usi=si;
```

执行上述两条语句后，usi的值为（ ）

- short为16位有符号数，类型的范围是 -32768 ~32767;
- unsigned short 为16位无符号数，类型的范围0~65535 。
- si=- -32767 对应的补码是 1000000000000001。
- Unsigned short usi=si; 此时usi的二进制码被解释为无符号数 1000 0000 0000 0001 ， 因此即为32769

>>> 2021、2022考研题

13. 已知带符号整数用补码表示，变量 x, y, z 的机器数分别为 FFFDH, FFDFH, 7FFCH，下列结论中，正确的是（ ）。

- A. 若 x, y 和 z 为无符号整数，则 $z < x < y$
- B. 若 x, y 和 z 为无符号整数，则 $x < y < z$
- C. 若 x, y 和 z 为带符号整数，则 $x < y < z$
- D. 若 x, y 和 z 为带符号整数，则 $y < x < z$

13. 32 位补码所能表示的整数范围是（ ）。

- A. $-2^{32} \sim 2^{31} - 1$
- B. $-2^{31} \sim 2^{31} - 1$
- C. $-2^{32} \sim 2^{32} - 1$
- D. $-2^{31} \sim 2^{32} - 1$

>>> 2023、2024考研题

13. 若 short 型变量 $x = -8190$ ，则 x 的机器数是 ()。

- A. E002H B. E001H C. 9FFFH D. 9FFE H

12. C语言代码段如下，执行该代码段后， j 的值是 ()。

```
int i=32777;  
short si=i;  
int j=si;
```

- A. -32777 B. -32759 C. 32759 D. 32777

408历年真题解析 <https://zhuanlan.zhihu.com/p/3484668199>

>>> 移码表示法

你能总结出移码的哪些规律？

➤ 移码：定点整数，一般用于表示浮点数的阶码；

➤ 定义：n+1位的移码表示为 $x_n x_{n-1} \dots x_1 x_0$

$$[x]_{\text{移}} = 2^n + x \quad -2^n \leq x < 2^n$$

由真值加一个固定的常数生成，这个固定的常数称为偏移量

□ 一般称为“移 2^n 码”；

✓ 如8位标准移码，称为移128码；

➤ 与补码关系：符号位相反，数值位相同；

➤ 优点：

□ 可以比较直观地判断两个数据的大小；

□ 表示浮点数阶码时，容易判断是否下溢；

✓ 当阶码为全0时，浮点数下溢。

4位补码与移码

真值	补码	移码
-8	1000	0000
-7	1001	0001
-6	1010	0010
.....		
0	0000	1000
+1	0001	1001
.....		
+7	0111	1111

>> 课本P29例2.8

将十进制真值(-127, -1, 0, +1, +127)列表表示成二进制数及原码、反码、补码、移码值。(移码8位表示, 偏移量为128)

十进制真值	二进制真值	原码表示	反码表示	补码表示	移码表示
-127	-111 1111	1111 1111	1000 0000	1000 0001	0000 0001
-1	-000 0001	1000 0001	1111 1110	1111 1111	0111 1111
0	+000 0000	0000 0000	0000 0000	0000 0000	1000 0000
	-000 0000	1000 0000	1111 1111		
+1	+000 0001	0000 0001	0000 0001	0000 0001	1000 0001
+127	+111 1111	0111 1111	0111 1111	0111 1111	1111 1111

>>> 原码、补码、移码

➤ 正数：

- 原、补码的编码完全相同；
- 补码和移码的符号位相反，数值位相同；

➤ 负数：

- 原码： 符号位为1
数值部分与真值的绝对值相同
- 补码： 符号位为1
数值部分与原码各位相反，且末位加1
- 移码： 符号位与补码相反，数值位与补码相同

➤ 注意：有些最值的求法不能用上述编码转换规则

>>> 课本P30例2.9

设机器字长16位，定点表示，数值15位，数符1位，问：

(1) 定点原码整数表示时，最大正数是多少？最小负数是多少？

(2) 定点原码小数表示时，最大正数是多少？最小负数是多少？

➤ 定点原码整数

□ 最大正数

0	111 1111 1111 1111
---	--------------------

$$(2^{15}-1) = +32767$$

□ 最小负数

1	111 1111 1111 1111
---	--------------------

$$-(2^{15}-1) = -32767$$

若用补码表示呢？
最小正数、最大
负数各是多少呢？

➤ 定点原码小数

□ 最大正数

0	111 1111 1111 1111
---	--------------------

$$(1-2^{-15}) = +(1-1/32768)$$

□ 最小负数

1	111 1111 1111 1111
---	--------------------

$$-(1-2^{-15}) = -(1-1/32768)$$

>>> 定点数表示方法

- **优点：**表示方法简单，便于运算
- **缺点：**表示的数的范围有限，要表示很大或很小的数必须用很多比特；s
数据精度低等
- **大数量级数据的表示**
 - 定点计算机间接表示：在运算之前先按照一定的固定比例（比例因子）缩放，在运算之后再恢复
 - 用幂的方式运算
 - 引入浮点数

>>> 2.1.1 数据格式——浮点数

课本P23

➤ 机器数的一般表示形式

$$N = (-1)^S \times R^e \times M$$

数符S	阶符	阶码e	尾数M
阶符	阶码e	数符S	尾数M

- 基数R：计算机内部数据默认R=2，不需要占用数据位表示；
- 数符S：1位，表示浮点数的正负；
- 尾数M：有符号定点纯小数，默认小数点在尾数之前；表示数据的全部有效数位，决定着数值的精度；
- 阶码e：比例因子的指数，又称浮点数的指数，用于指出小数点在该数中的位置，决定着数据数值的大小。有符号定点纯整数，包含符号位（阶符）和数值位；
- 一个浮点数就可以有两个定点数表示
 - ✓ M称为浮点数的尾数，用一个定点小数来表示；
 - ✓ E称为浮点数的指数或阶码，用一个定点整数来表示。

>>> 浮点数举例

➤ 请将数据-12.625D用16位的浮点数形式表示。

□ 数据格式如下，阶码（含阶符）4位，尾数（含数符）12位；

阶符	阶码	数符	尾数
1位	3位	1位	11位

➤ $-12.625D = -1100.101 B = -0.1100 101 \times 2^4 B$

➤ 设尾数用原码表示，阶码用补码表示；

□ 浮点数表示为： 0 100 ; 1.110 0101 0000

➤ 设尾数用补码表示，阶码用移码表示；

□ 浮点数表示为： 1 100 ; 1.001 1011 0000

>>> 浮点数规格化

➤ 浮点数的一般表示

□ 同一个浮点数的表示是不唯一的。如 0.1101 可表示为 0.01101×2^1 ，也可表示为 1.101×2^{-1} 。

□ 机器数的表示不同，不利于运算。

➤ 规格化：是浮点数唯一形式的表示；

➤ 规格化的目的

□ 保证浮点数表示的唯一性；

□ 保留更多地有效数字，提高运算的精度。

>>> 浮点数规格化

➤规格化要求：当尾数的值不为0时， **$|\text{尾数}| \geq 0.5$** ；

□尾数**原码**表示：最高数值位为1，正数0.1...，负数1.1...；

✓例如， 0.011×2^5 要规格化则变为 0.11×2^4 ；

- 0.011×2^5 要规格化则变为 1.11×2^4 ；

高位上的0代表实际
绝对值对应位上的1

□尾数**补码**表示：最高数值位和符号位相反，正数0.1...，负数1.0...；

✓ $[+0.1B]_{\text{补}} = 0.10 \cdots 0$ $[-0.1B]_{\text{补}} = 1.10 \cdots 0$ $[-0.10 \cdots 01B]_{\text{补}} = 1.01 \cdots 11$

➤规格化处理：

□尾数向左移n位（小数点右移），同时阶码减n；——→ 左规

□尾数向右移n位（小数点左移），同时阶码加n。——→ 右规

>>> 浮点数规格化-----分析 (负数补码)

➤规格化要求：当尾数的值不为0时，**|尾数| ≥ 0.5**；

□尾数**补码**表示：最高数值位和符号位相反，正数0.1..., 负数1.0...；

➤按照规格化要求和尾数为纯小数两个条件，负数M范围应该是： **$-1 < M \leq -\frac{1}{2}$**

□由上可知M补码表示，按照补码定义

定义：	$[x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 2+x = 2 - x & 0 \geq x \geq -1 \end{cases} \pmod{2}$
○定点小数：	

$$-1+2 < M+2 \leq 2+(-\frac{1}{2}) \quad \longrightarrow \quad 1 < M+2 \leq 1\frac{1}{2} \quad \longrightarrow \quad 1 < [M]_{\text{补}} \leq 1\frac{1}{2}$$

>>> 浮点数规格化-----分析（负数补码）规格化通式

➤按照规格化要求和尾数为纯小数两个条件，负数M范围应该为： $-1 < M \leq -\frac{1}{2}$

□M补码表示，按照定义 $\longrightarrow 1 < [M]_{\text{补}} \leq 1\frac{1}{2}$

真值	二进制表示
-1/2:	1.100*****00
	1.011*****11
	1.011*****10
	
	1.000*****01
-1 :	1.000*****00

-1原本不在补码表示范围，但 $-\frac{1}{2}$ 与其他的数据1.0*****表示又不同，为了统一表示，计算机判断方便（例如异或电路实现符号判断），M补码表示为（去 -1/2，添加-1）：



尾数（负数）M规格化的表示应为：1.0*****

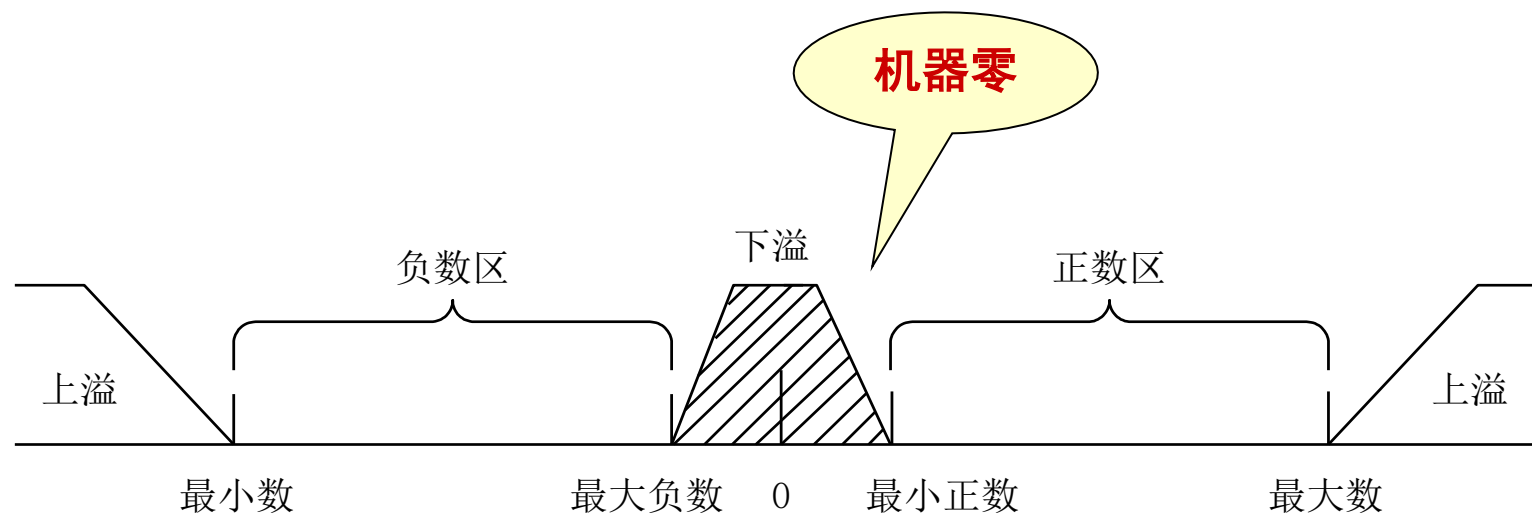
即 $M < 0$ ， $[-1]_{\text{补}} \leq [M]_{\text{补}} < [-1/2]_{\text{补}}$

>>> 浮点数的数据表示范围----机器零

➤ 在浮点数的表示范围中，有两种情况被称为机器零：

- (1) 若浮点数的尾数为零，无论阶码为何值；
- (2) 当阶码的值遇到比它能表示的最小值还要小时，无论其尾数为何值

➤ 机器零：浮点数的尾数为零或阶码小于机器能表示的最小阶码



>>> 浮点数的溢出

➤ 浮点数的溢出：数的大小超出了浮点数的表示范围

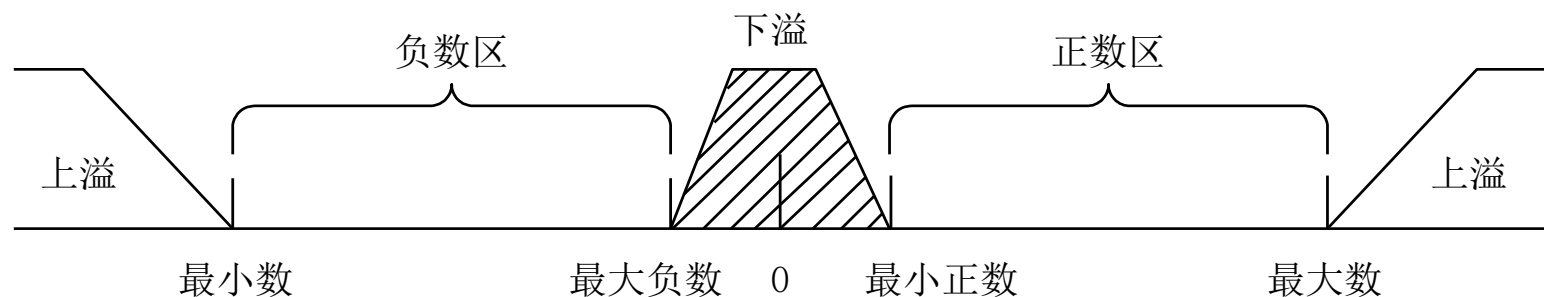
□ 原因：指数部分太大，无法用有限的指数字段表示出来

✓ 下溢：阶码小于机器能表示的最小阶码—— 一般作为机器零处理

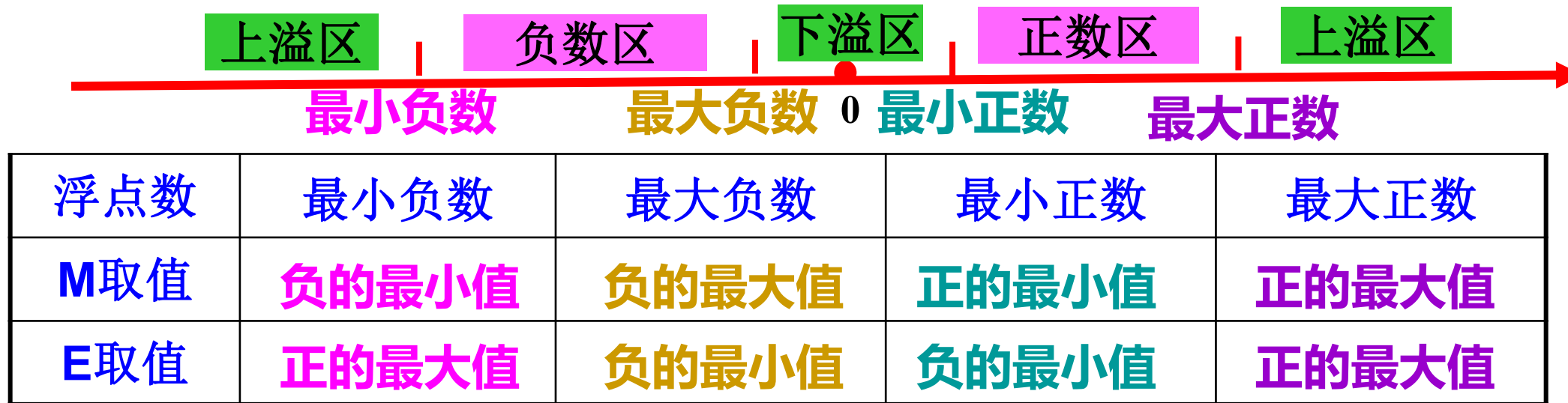
✓ 上溢：阶码大于机器能表示的最大阶码—— 必须转为中断处理

➤ 如何减少溢出

□ 双精度格式



>>> 浮点数的数据表示范围



➤ 浮点数的溢出：阶码溢出

□ 上溢：阶码大于所能表示的最大值；

□ 下溢：阶码小于所能表示的最小值；

➤ 机器零：尾数为 0，或阶码小于所能表示的最小值；

>>> 定点数与浮点数的数据表示范围和精度比较

➤ 定点数

以16位数据为例，
补码表示

□ 定点整数： $-2^{15} \sim +2^{15}-1$ (即 $-32768 \sim +32767$)

□ 定点小数： $-1 \sim +1-2^{-15}$ (即 $-1 \sim +0.999\ 9694\ 8242\ 1875$)

➤ 浮点数

1位阶符	7位阶码	1位数符	7位尾数
------	------	------	------

□ 格式：

□ 最大正数： $+127 \times 2^{+127}B$

□ 最小正数： $+0.0000001 \times 2^{-127}B$

□ 最大负数： $-0.0000001 \times 2^{-127}B$

□ 最小负数： $-127 \times 2^{+127}B$

>>> 务必牢记

- 浮点数的表示要**与具体的格式规定有关**;
- 做题时, 要看清题目里要求的浮点数各部分的**位数、排列和编码形式**;
- 一般机器中, 浮点数采用IEEE754标准来存放float、double类型的变量;
 - IEEE754标准只是浮点数的一种表示形式;

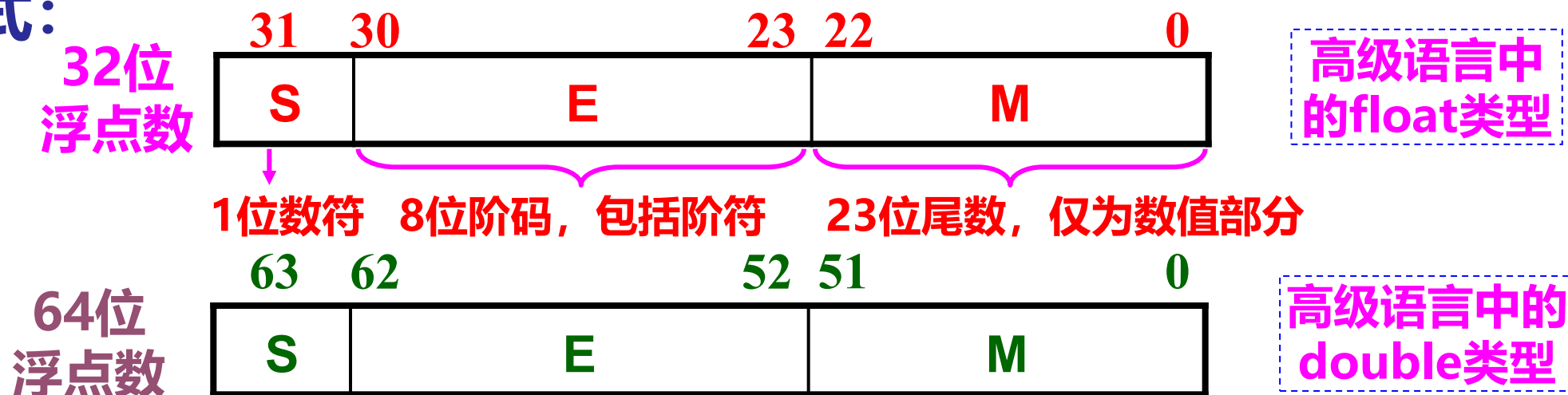
>>> 浮点数的IEEE754标准表示

➤ IEEE (Institute of Electrical and Electronics Engineers)

□ 美国电气及电子工程师学会

□ IEEE致力于电气、电子、计算机工程和与科学有关的领域的开发和研究，也是计算机网络标准的主要制定者。

➤ 为便于软件移植，IEEE754 标准规定了计算机内部32位/64位浮点数的标准格式：



>>> 32位浮点数的IEEE754 标准表示及其规定

数符S	阶码E (8位移码)	尾数M (23位原码)
-----	------------	-------------

- 数符S：表示浮点数的符号，占1位，0—正数、1—负数；
- 尾数M：23位，原码纯小数表示，小数点在尾数域的最前面；
 - 由于原码表示的规格化浮点数要求，最高数值位始终为1，因此该标准中隐藏最高数值位(1)，尾数的实际值为1.M；
- 阶码E：8位，采用有偏移值的移码表示；
 - 移127码，即 $E = e + (2^7 - 1) = e + 127$ ；
 - ✓ 标准8位移码应该是移128码， $[x]_{\text{移}} = x + 2^7 = x + 128$
- 浮点数的真值： $N = (-1)^S \times (1.M) \times 2^{E-127}$
- 指数（阶码）的最大值和最小值作为特殊预留，用来表示异常事件或机器零。
- 实际精度24位（加上隐藏位）

e 的取值范围为-127 ~ +128



754标准: $[X]_{\text{移}} = 2^{m-1} + X$, 127

最后将移码看做无符号数
注意只是看做而已

真值	原码		补码	移码	无符号数
-128			1000 0000	1111 1111	255
-127	1111 1111		1000 0001	0000 0000	0
-126	1111 1110		1000 0010	0000 0001	1
...	...		...	...	...
0	1000 0000		0000 000	0111 1111	127
1	0000 0001		0000 0001	1000 0000	128
...	...		...	...	...
126	0111 1110		0111 1110	1111 1101	253
+127	0111 1111		0111 1111	1111 1110	254

这两个特殊

>>> IEEE754 标准的数据表示

➤ IEEE754 标准中的阶码E 0000 0000 ~ 1111 1111

□ 正零、负零 E=0000 0000(真值-127), M=0000 ... 0000

✓ E与M均为零, 真值 $N=(-1)^s \times 0$, 机器0, 正负之分由数据符号确定;

□ 正无穷、负无穷 E=1111 1111 (真值-128), M=0000 ... 0000

✓ E为全1, M为全零, 真值 $N=(-1)^s \times \infty$, 正负之分由数据符号确定;

□ 阶码E的其余值 (0000 0001~1111 1110) 为规格化数据;

✓ E为1 ~ 254, 真正的指数e的范围为-126 ~ +127 ($e=E-127$)

➤ 为避免浮点数下溢, 尾数非0时, 阶码为0时, 允许采用比最小规格化数还小的非规格化数来表示, 但此时尾数M前的隐含位为0, 而不是1。

✓ 真值 $N=(-1)^s \times (0.M) \times 2^{-126}$ (非规格化)

>>> 课本P23 例2.5

例1. 若浮点数 x 的754标准存储格式为 $(41360000)_{16}$, 求其浮点数的十进制数

解:

$$\square (41360000)_{16} = \underset{\substack{\downarrow \\ \text{数符} S}}{0} \underbrace{100\ 0001\ 0}_{\text{阶码 } E} \underbrace{0111\ 0110\ 0000\ 0000\ 0000\ 0000}_{\text{尾数 } M}$$

$$\square \text{指数 } e = E - 127 = 1000\ 0010 - 0111\ 1111 = 0000\ 0011 = 3$$

$$\square \text{尾数 } 1.M = 1.011\ 0110\ 0000\ 0000\ 0000\ 0000 = 1.011011$$

$$\begin{aligned} \square \text{浮点数 } N &= (-1)^S \times (1.M) \times 2^e \\ &= (-1)^0 \times (1.011011) \times 2^3 \\ &= (11.375)_{10} \end{aligned}$$

>>> 课本P29 例2.6

例2. 将 $(20.59375)_{10}$ 转换成754标准的32位浮点数的二进制存储格式。

解：

□ $(20.59375)_{10} = (10100.10011)_2$

□ 将尾数规范为1.M的形式：

$$10100.10011 = 1.010010011 \times 2^4 \quad e = 4$$

□ 可得：M = 010010011

$$S = 0$$

$$E = 4 + 127 = 131 = 1000\ 0011$$

□ 故，32位浮点数的754标准格式为：

$$0100\ 0001\ 1010\ 0100\ 1100\ 0000\ 0000\ 0000 = (41A4C000)_{16}$$

>>> 练习

将下列十进制数表示成IEEE754格式的32位浮点数形式存储。

(1) 27/32

(2) 11/512

解:

$$(1) \ 27/32 = 27 * (1/32) = (0001 \ 1011)_2 * 2^{-5}$$

✓ 尾数: 1.1011; 阶码: $e = -5 + 4 = -1$, $E = e + 127 = 126$

✓ IEEE754数据

0 0111 1110 1011 0000 0000 0000 0000 000

$$(2) \ 11/512 = (0000 \ 1011)_2 * 2^{-9}$$

✓ 尾数: 1.011 ; 阶码: $e = -9 + 3 = -6$, $E = e + 127 = 121$

✓ IEEE754数据

0 0111 1001 0110 0000 0000 0000 0000 00

>>> 2011年考研统考题目 第13题

➤ float型数据通常用IEEE754单精度浮点数格式表示。若编译器将float型变量x分配到一个32位浮点寄存器FR1中，且 $x = -8.25$ ，则FR1的内容是（ A ）。

A. C104 0000H

B. C242 0000H

C. C184 0000H

D. C1C2 0000H

解答：

$$X = -8.25 = -1000.01 = -1.00001 \times 2^3$$

$$\therefore S=1 \quad M=000010\cdots0 \quad E=127+3=1000\ 0010$$

数符S	阶码E (8位移码)	尾数M (23位原码)
1	100 0001 0	000 0100 0000 0000 0000 0000

>>> 2012年考研统考题目 第14题

➤ 14.float类型（即IEEE 754单精度浮点数格式）能表示的最大正整数是（ ）

A. $2^{126}-2^{103}$

B. $2^{127}-2^{104}$

C. $2^{127}-2^{103}$

D. $2^{128}-2^{104}$

解答：

最大正整数： $1.M \times 2^{E-127}$ E的有效最大值为254

$$1.M \times 2^{E-127} = (1 + 1 - 2^{-23}) \times 2^{254-127}$$

$$= 2 \times 2^{127} - 2^{-23+127}$$

$$= 2^{128} - 2^{104}$$

2013年考研统考题目 第13题

13. 某数采用IEEE 754单精度浮点数格式表示为C640 0000H，则
该数的真值为（ ）

A -1.5×2^{13}

B -1.5×2^{12}

C -0.5×2^{13}

D -0.5×2^{12}

提交

>>> 2013年考研统考题目 第13题

13. 某数采用IEEE 754单精度浮点数格式表示为C640 0000H, 则该数的真值为 () **A**

A. -1.5×2^{13}

B. -1.5×2^{12}

C. -0.5×2^{13}

D. -0.5×2^{12}

解答:

➤ 机器数: **1100 0110 0100 0000 0000**

➤ $S=1$, $E=10001100$, $M=1.1$

➤ $N = (-1)^1 \times 1.1 + 1101 = -1.5 \times 2^{13}$

2020年考研统考题目 第13题

13. 已知带符号整数用补码表示，float型数据用IEEE754标准表示，假定变量x的类型只能是int或float。当x的机器数为C800 0000H时，x的值可能是（ ）

☒ A -7×2^{27}

☐ B -2^{16}

☐ C 2^{17}

☐ D 25×2^{27}

提交

2020年考研统考题目 第13题

13. 已知带符号整数用补码表示，float型数据用IEEE754标准表示，假定变量x的类型只能是int或float。当x的机器数为C800 0000H时，x的值可能是（ ）

E														
D ₃₁	D ₃₀	D ₂₉	D ₂₈	D ₂₇	D ₂₆	D ₂₅	D ₂₄	D ₂₃	~D ₂₀	D ₁₉ ~D ₁₆	D ₁₅ ~D ₁₂	D ₁₁ ~D ₈	D ₇ ~D ₄	D ₃ ~D ₀
1	1	0	0	1	0	0	0	0	0	0	0	0	0	0
-	0	1	1	1	0	0	0							

➤ 若为float数据，则s=1，E=1001 0000，M=1.0

$$\square x = -1 \times 2^{17}$$

➤ 若为int数据，按补码规则，其真值为-01110...0

$$\square x = -(2^{29} + 2^{28} + 2^{27}) = -7 \times 2^{27}$$

>>> 2021、2022年考研题

13. 已知带符号整数用补码表示, 变量 x, y, z 的机器数分别为 FFFDH, FFDFH, 7FFCH, 下列结论中, 正确的是 ()。

- A. 若 x, y 和 z 为无符号整数, 则 $z < x < y$
- B. 若 x, y 和 z 为无符号整数, 则 $x < y < z$
- C. 若 x, y 和 z 为带符号整数, 则 $x < y < z$
- D. 若 x, y 和 z 为带符号整数, 则 $y < x < z$

14. 下列数值中, 不能用 IEEE 754 浮点格式精确表示的是 ()。

- A. 1.2
- B. 1.25
- C. 2.0
- D. 2.5

13. 32 位补码所能表示的整数范围是 ()。

- A. $-2^{32} \sim 2^{31} - 1$
- B. $-2^{31} \sim 2^{31} - 1$
- C. $-2^{32} \sim 2^{32} - 1$
- D. $-2^{31} \sim 2^{32} - 1$

14. -0.4375 的 IEEE 754 单精度浮点数表示为 ()。

- A. BEE0 0000H
- B. BF60 0000H
- C. BF70 0000H
- D. C0E0 0000H

>>> 2023、2024、2025年考研题

14. 已知 float 型变量用 IEEE 754 单精度浮点数格式表示。若 float 型变量 x 的机器数为 8020 0000H，则 x 的值是（ ）。

- A. -2^{-128} B. -1.01×2^{-127} C. -1.01×2^{-126} D. 非数 (NaN)

14. 某科学实验中，需要使用大量的整型参数，为了保证表数精度的基础上提高运算速度，需要选择合理的数据表示方法。若整型参数 α 、 β 的取值范围分别为 $-2^{20} \sim 2^{20}$ 、 $-2^{40} \sim 2^{40}$ ，则下列选项中， α 、 β 最适宜采用的数据表示方法分别是（ ）。

- A. 32位整数、32位整数 B. 单精度浮点数、单精度浮点数
C. 32位整数、双精度浮点数 D. 单精度浮点数、双精度浮点数

13. 在 IEEE754 标准下，4730 0000H 对应的真值是多少（ ）。

- A. 0.375×2^{14} B. 1.6375×2^{14}
C. 0.375×2^{15} D. 1.375×2^{15}

【参考答案】D



课本P30例2.10
(类似IEEE754标准的公式，但阶码采用的是移128码)

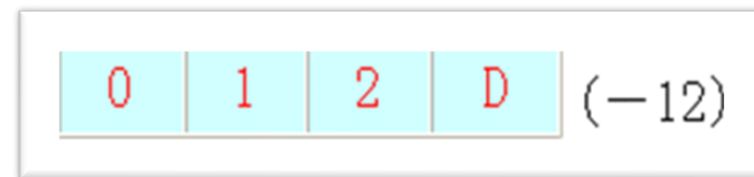
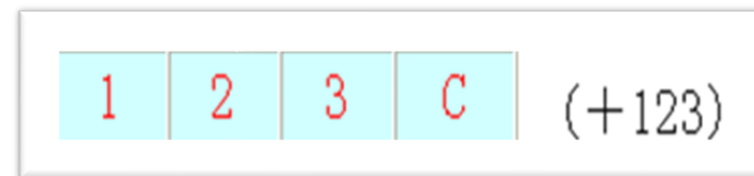
>>> 2.1.3 字符与字符串的表示方法 (1/2)

➤ 字符串形式

- 每个十进制**数位**占用一个字节；
- 除保存各数位，还需要指明该数存放的起始地址和总位数；
- 主要用于非数值计算的应用领域。

➤ 压缩的十进制数串形式

- 采用BCD码表示，一个字节可存放两个十进制数位；
- 节省存储空间，便于直接完成十进制数的算术运算；
- 用特殊的二进制编码表示数据正负，如1100—正、1101—负；



>>> 2.1.3 字符与字符串的表示方法 (2/2)

➤ ASCII码

(美国国家信息交换标准字符码)

- 包括128个字符，共需7位编码；
- ASCII码**最高位为0**，余下7位作为128个字符的编码。
- 最高位的作用
 - ✓ 奇偶校验；扩展编码。

➤ 常用ASCII码如右表：

字符	ASCII码（十六进制表示）
‘0’~‘9’	30H ~ 39H
‘a’~‘z’	61H ~ 7AH
‘A’~‘Z’	41H ~ 5AH
空格	20H
‘\$’	24H
回车	0DH
换行	0AH

>>> 2.1.5 校验码（数据校验）

- 数据校验：由数据特定编码形式，检查和纠正数据传送过程中出现的错误；
- 数据校验码（检错码）：能够发现某些错误或具有自动纠错能力的数据编码；
- 数据校验的原理：扩大码距；

□ 码距：任意两个合法码之间不同的二进制位的最少位数；

➤ 设有四位二进制编码

- 若表示16种状态，则任何一位出错，即变成另一个合法码；
 - ✓ 此时码距为1，无检错能力；
- 若表示8种状态（编码设置合适），则一位出错变为非法编码；
 - ✓ 此时码距可以扩大为2；
 - ✓ 注意：并不是任选8种编码都可扩大码距；

例如，
码距为2的4位编码
0000、0011、0101、
0110、1001、1010、
1100、1111

>>> 补码表示法的引入

➤ 继续推导:

$$\square 5 - 2 = 5 + 10 \pmod{12}$$

$$\square 5 + (-2) = 5 + 10 \pmod{12}$$

$$\square -2 = 10 \pmod{12} \quad \text{注意在模12的条件下}$$

➤ 结论:

在模为**12**的情况下，**-2**的补码就是**10**。一个负数用其补码代替，同样可以得到正确的运算结果。

>>>补码表示法的引入

➤进一步结论：

□在计算机中，机器能表示的数据位数是固定的，其运算都是有模运算。

✓若是 $n+1$ 位整数，则其模为 2^{n+1} ；

✓若是小数，则其模为2。

□若运算结果超出了计算机所能表示的数值范围，则只保留它的小于模的低 n 位的数值，超过 n 位的高位部分就自动舍弃了（联想纯小数补码绝对值最大的-1）。

>>>补码表示法----定义

➤定义:

□定点小数: $[x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 2+x = 2 - |x| & 0 \geq x \geq -1 \end{cases} \quad (\text{mod } 2)$

□定点整数: $[x]_{\text{补}} = \begin{cases} x & 2^n > x \geq 0 \\ 2^{n+1}+x = 2^{n+1}-|x| & 0 \geq x \geq -2^n \end{cases} \quad (\text{mod } 2^{n+1})$

➤举例: x为n+1位

□ $[+0.110]_{\text{补}} = 0.110$

□ $[-0.110]_{\text{补}} = 10 + (-0.110) = 1.010$

□ $[+110]_{\text{补}} = 0110$

□ $[-110]_{\text{补}} = 2^4 + (-110) = 10000 - 110 = 1010$

2.2 定点加法、减法运算

4.2.1 补码加法

4.2.2 补码减法

4.2.3 溢出概念与检验方法

4.2.4 基本的二进制加法、减法器

>>> 定点数补码加法举例

[例2.12] $x = +1001$, $y = +0101$,

求 $x + y$ 。

解:

$$[x]_{\text{补}} = 0\ 1001, [y]_{\text{补}} = 0\ 0101$$

$$\begin{array}{r} [x]_{\text{补}} \quad 0\ 1001 \\ + [y]_{\text{补}} \quad 0\ 0101 \\ \hline [x + y]_{\text{补}} \quad 0\ 1110 \end{array}$$

$$\therefore x + y = +1110$$

[例2.13] $x = +1011$, $y = -0101$,

求 $x + y$ 。

解:

$$[x]_{\text{补}} = 0\ 1011, [y]_{\text{补}} = 1\ 1011$$

$$\begin{array}{r} [x]_{\text{补}} \quad 0\ 1011 \\ + [y]_{\text{补}} \quad 1\ 1011 \\ \hline [x + y]_{\text{补}} \quad 10\ 0110 \end{array}$$

$$\therefore x + y = +0110$$

>>> 补码负的最小值的计算

$$[-127]_{\text{补}} = [-111\ 1111]_{\text{补}} = 1000\ 0001$$

$$[-1]_{\text{补}} = [-000\ 0001]_{\text{补}} = 1111\ 1111$$

$$11000\ 0000$$

$$\text{mod } 2^8$$

$$1000\ 0000 = [-128]_{\text{补}}$$

>>> 2.2.2 补码减法

➤ 补码减法运算基本公式

□ 定点整数: $[x - y]_{\text{补}} = [x]_{\text{补}} - [y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}} \pmod{2^{n+1}}$

□ 定点小数: $[x - y]_{\text{补}} = [x]_{\text{补}} - [y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}} \pmod{2}$

➤ 证明: 只需要证明 $[-y]_{\text{补}} = -[y]_{\text{补}}$

□ 已证明 $[x+y]_{\text{补}} = [x]_{\text{补}} + [y]_{\text{补}}$, 故 $[y]_{\text{补}} = [x+y]_{\text{补}} - [x]_{\text{补}}$

□ 又 $[x - y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$, 故 $[-y]_{\text{补}} = [x - y]_{\text{补}} - [x]_{\text{补}}$

□ 可得 $[y]_{\text{补}} + [-y]_{\text{补}} = [x+y]_{\text{补}} + [x - y]_{\text{补}} - [x]_{\text{补}} - [x]_{\text{补}}$
 $= [x + y + x - y]_{\text{补}} - [x]_{\text{补}} - [x]_{\text{补}}$
 $= [x + x]_{\text{补}} - [x]_{\text{补}} - [x]_{\text{补}} = 0$

➤ $[-y]_{\text{补}}$ 等于 $[y]_{\text{补}}$ 的各位取反, 末位加1。

>>> 定点数补码减法举例：课本P36 例2.14

已知 $x_1 = -1110$, $x_2 = +1101$,

求： $[x_1]_{\text{补}}$, $[-x_1]_{\text{补}}$, $[x_2]_{\text{补}}$, $[-x_2]_{\text{补}}$ 。

➤解：

$$[x_1]_{\text{补}} = 1\ 0010$$

$$\begin{aligned} [-x_1]_{\text{补}} &= \neg [x_1]_{\text{补}} + 1 \\ &= 0\ 1101 + 0\ 0001 = 0\ 1110 \end{aligned}$$

$$[x_2]_{\text{补}} = 0\ 1101$$

$$\begin{aligned} [-x_2]_{\text{补}} &= \neg [x_2]_{\text{补}} + 1 \\ &= 1\ 0010 + 0\ 0001 = 1\ 0011 \end{aligned}$$

注意课本这里
为 2^{-4} ，是将其
作为定点小数
处理！

>>> 定点数补码减法举例：课本P37 例2.15

$x = +1101$, $y = +0110$, 求 $x - y$ 。

➤解:

$$[x]_{\text{补}} = 0\ 1101, [y]_{\text{补}} = 0\ 0110, [-y]_{\text{补}} = 1\ 1010$$

$$[x - y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$$

$$= 0\ 1101 + 1\ 1010$$

$$= 10\ 0111$$

$$= 0\ 0111$$

$$\therefore x - y = +0111$$

$$\begin{array}{r} 0\ 1101 \\ +) 1\ 1010 \\ \hline \boxed{1}\ 0\ 0111 \end{array}$$

>>> 定点数补码加减法运算

➤ 基本公式

□ 定点整数:

$$[x \pm y]_{\text{补}} = [x]_{\text{补}} + [\pm y]_{\text{补}} \pmod{2^{n+1}}$$

□ 定点小数:

$$[x \pm y]_{\text{补}} = [x]_{\text{补}} + [\pm y]_{\text{补}} \pmod{2}$$

➤ 定点数补码加减法运算

□ 符号位和数值位可同等处理;

□ 只要结果不溢出, 将结果按 2^{n+1} 或2取模, 即为本次运算结果。

例. 设机器字长为8位, $[x]_{\text{补}} = 1010\ 0011$, $[y]_{\text{补}} = 0010\ 1101$,
求 $x - y$ 。

作答

>>> 例 设机器字长为8位, $[x]_{\text{补}} = 1010\ 0011$, $[y]_{\text{补}} = 0010\ 1101$,
求 $x - y$.

$x = -93$, $y = +45$

解:

$$[-y]_{\text{补}} = 1101\ 0011$$

$$[x - y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$$

$$= 1010\ 0011 + 1101\ 0011$$

$$= 1\ 0111\ 0110$$

$$= 0111\ 0110$$

$$\therefore x - y = +118$$

$$\begin{array}{r} 1010\ 0011 \\ +) 1101\ 0011 \\ \hline 1\ 0111\ 0110 \end{array}$$

计算过程中, 产生了溢出!

$$-93 - 45 = -138 < -128$$

>>> 2.2.3 溢出概念与检测方法

➤ 补码加法运算先决条件: $|x+y| < 1$

➤ 溢出

□ 定点数机中, 数值大小超出数据所能表示的范围。

➤ 正溢 (上溢)

□ 数据大于机器所能表示的最大正数;

➤ 负溢 (下溢)

□ 数据小于机器所能表示的最小负数;

➤ 例如, 4位补码表示的定点整数, 范围为 $[-8, +7]$

□ 若 $x = 5, y = 4$, 则 $x+y$ 产生上溢

□ 若 $x = -5, y = -4$, 则 $x+y$ 产生下溢

□ 若 $x = 5, y = -4$, 则 $x-y$ 产生上溢

例. $x = +0.1011, y = +0.1001$,

求 $x + y$ 。

[解:] $[x]_{\text{补}} = 0.1011$

$[y]_{\text{补}} = 0.1001$

$$\begin{array}{r} [x]_{\text{补}} \quad 0.1011 \\ + [y]_{\text{补}} \quad 0.1001 \\ \hline \end{array}$$

$[x+y]_{\text{补}} = 1.0100$

正数+正数=负数

溢出!

例. $x = -0.1101, y = -0.1111$,

求 $x + y$ 。

[解:] $[x]_{\text{补}} = 1.0011$

$[y]_{\text{补}} = 1.0001$

$$\begin{array}{r} [x]_{\text{补}} \quad 1.0011 \\ + [y]_{\text{补}} \quad 1.0001 \\ \hline \end{array}$$

$[x+y]_{\text{补}} = 0.0101$

负数+负数=正数

>>> 溢出判别方法1——直接判别法

➤ 方法:

□ 同号补码相加, 结果符号位与加数相反;

□ 异号补码相减, 结果符号位与减数相同;

➤ 硬件实现较复杂: x 和 y 的符号位异或后, 再与结果符号位异或;

➤ 举例:

□ 若 $[x]_{\text{补}}=0101$, $[y]_{\text{补}}=0100$, 则 $[x+y]_{\text{补}}=1001$ 上溢

□ 若 $[x]_{\text{补}}=1011$, $[y]_{\text{补}}=1100$, 则 $[x+y]_{\text{补}}=0111$ 下溢

□ 若 $[x]_{\text{补}}=0101$, $[y]_{\text{补}}=1100$, 则 $[x-y]_{\text{补}}=1001$ 上溢

>>> 溢出判别方法2——变形补码判别法(双符号位)

➤ 变形补码（模4补码）：

□ 采用双符号位表示补码，00-正数，11-负数；

□ 若运算结果双符号位相反，则溢出；

✓ 01：正溢，大于最大正数；

✓ 10：负溢，小于最小负数；

双符号位	结果
00	正
01	正溢
10	负溢
11	负

➤ 优点：硬件实现简单，只需对结果的双符号位进行异或；

➤ 举例：

□ 若 $[x]_{\text{补}} = 00101$ ， $[y]_{\text{补}} = 00100$ ，则 $[x+y]_{\text{补}} = 01001$

正溢

□ 若 $[x]_{\text{补}} = 11011$ ， $[y]_{\text{补}} = 11100$ ，则 $[x+y]_{\text{补}} = 10111$

负溢

□ 若 $[x]_{\text{补}} = 00101$ ， $[y]_{\text{补}} = 11100$ ，则 $[x-y]_{\text{补}} = 01001$

正溢

>>> 溢出判别方法3——进位判别法（单符号）

➤ 判别方法：

□ 最高数值位的进位与符号位的进位是否相同；

➤ 判别公式

□ 溢出标志 $OF = C_f \oplus C_{n-1}$

✓ C_f 为符号位产生的进位；

✓ C_{n-1} 为最高数值位产生的进位。

➤ 硬件实现简单，只需对两个进位位进行异或；

➤ 现代处理器采用这种方法

$$\begin{array}{r} 0101 \\ +) 0100 \\ \hline \end{array} \quad OF=0\oplus 1=1$$

1001
↕
0 1

$$\begin{array}{r} 0001 \\ +) 0100 \\ \hline \end{array} \quad OF=0\oplus 0=0$$

0101
↕
0 0

2014年考研统考题目 第13题

若 $x=103$ ， $y=-25$ ，则下列表达式采用8位定点补码运算实现时，会发生溢出的是（ ）。

☐ A $x+y$

☐ B $-x+y$

☒ C $x-y$

☐ D $-x-y$

提交

2014年考研统考题目 第13题

若 $x=103$ ， $y=-25$ ，则下列表达式采用8位定点补码运算实现时，会发生溢出的是（ ）。

解：同号相加或异号相减才有可能溢出，排除A和D

8位定点整数范围-128----- +127

A $x+y$

B $-x+y$

C $x-y$

D $-x-y$

2013年考研统考题目 第14题

某字长为8位的计算机中，已知整型变量x、y的机器数分别为

$$[x]_{\text{补}} = 1111\ 0100$$

$$[y]_{\text{补}} = 1011\ 0000$$

若整型变量 $z = 2 * x + y / 2$ ，则z的机器数为（ ）

A

1 1000000

B

0010 0100

C

1010 1010

D

溢出

提交

>>> 2013年考研统考题目 第14题

某字长为8位的计算机中，已知整型变量x、y的机器数分别为

$$[x]_{\text{补}} = 1111\ 0100$$

$$[y]_{\text{补}} = 1011\ 0000$$

若整型变量 $z = 2 * x + y / 2$ ，则z的机器数为（ ）

- $x * 2$, 将x 算术左移一位，用于带符号数左移，符号位不变，结果为1 1101000;
- $y / 2$, 将y 算术右移一位，同理符号位保持不变，右移后的空位用符号位补充，结果为1 1011000, 均无溢出或丢失精度。补码相加为 $1\ 1101000 + 1\ 1011000 = 1\ 1000000$, 亦无溢出。
- 在不溢出的结果下等于乘以2或除以2

2.6 浮点运算方法和浮点运算器

2.6.1 浮点加法、减法运算

2.6.2 浮点乘法、除法运算

2.6.3 浮点运算流水线

2.6.4 浮点运算器实例

>>> 2.6.1 浮点数加减运算

- 设有两个浮点数 $X = M_x \times 2^{E_x}$ 和 $Y = M_y \times 2^{E_y}$ ，且 $E_x > E_y$
- 若要求 $X \pm Y$ 的结果 S ，则
 - $S = X \pm Y = M_s \times 2^{E_s}$ 其中， $E_s = E_x$ ， $M_s = M_x \pm (M_y \text{ SHR } (E_x - E_y))$
- 浮点数加减运算的步骤
 - 零操作数检查 一个操作数为0，则不必运算，节省运算时间
 - 两操作数对阶 小数点位置对齐，即阶码相同，为加减运算做准备
 - 尾数相加减 定点小数相加减，注意尾数的溢出判断
 - 结果的规格化 将非规格化的结果值进行规格化处理
 - 结果的舍入处理 尽量保持计算后数据的精确度
 - 结果的溢出判断 以阶码溢出为标准，尾数溢出则继续规格化处理

>>> 浮点数加减运算——两操作数对阶

➤ 对阶的原则

□ 以较大的阶码为标准，调整阶码较小的数据；

✓ 避免阶码较大的浮点数的尾数左移，导致最高有效数位丢失；

增大阶码，尾数右移

➤ 具体操作

□ 求阶差 $\Delta E = E_X - E_Y$

□ 调整阶码较小的数据

✓ 若 $\Delta E > 0$ ，则 M_Y 右移 ΔE 位，结果的阶码为 E_X

✓ 若 $\Delta E < 0$ ，则 M_X 右移 $|\Delta E|$ 位，结果的阶码为 E_Y

➤ 例，X—— $E_X = 0\ 001$ ， $M_X = 0.101$ ；Y—— $E_Y = 0\ 011$ ，尾数 $M_Y = 0.111$

□ 阶差 $\Delta E = E_X - E_Y = 0\ 001 - 0\ 011 = -10 < 0$

□ X尾数 M_X 右移2位， $M_X = 0.001\ 01$

>>> 浮点数加减运算——结果的规格化处理

➤ 两尾数加减的结果有两种情况

□ 尾数溢出 $\longrightarrow$ 两符号位为01或10 $\longrightarrow$ 右规

✓ 尾数右移1位，阶码加1

□ 尾数为非规格化数据 $\longrightarrow$ 补码表示的符号位与最高数值位相同 $\longrightarrow$ 左规

✓ 尾数左移1位，阶码减1，直至数值位最高位与符号位相反。

➤ 接上例，对阶后 $E=E_Y=0\ 011$ ，尾数 $M_X=0.001\ (01)$ ， $M_Y=0.111$

□ 尾数求和： $M_S = M_X + M_Y = 00.001\ 01 + 00.111 = 01.000\ 01$

□ 两符号位相反，应进行右规1位的操作

□ 则 $M_S = 00.100\ (001)$ ， $E_S = 011+1 = 0\ 100$

>>> 浮点数加减运算——结果的舍入处理

➤ 在对阶或右规操作时，会使加数或结果的尾数低若干位移出，影响精度，常用两种舍入处理方法(注意教材上的4种方法)

□ 方法1：0舍1入法

- ✓ 保留右移时的移出位，若最高位为1，则尾数加1；否则舍去；
- ✓ 特点：精度较高，但需要记录所有的移出位。

□ 方法2：恒置1法

- ✓ 若之前步骤有右移操作，则直接将结果的最低位置1；
- ✓ 特点：精度较0舍1入法较低，但应用简单。

➤ 接上例，结果的尾数 $M_S = 00.100\ 001$

□ 0舍1入法： $M_S = 00.100$

□ 恒置1法： $M_S = 00.101$

>>> 浮点数加减运算——结果的溢出判断

➤ 尾数溢出

□ 在规格化处理时，通过完成右规完成；

➤ 阶码溢出

□ 上溢(结果绝对值太大)——置上溢标志，结束；

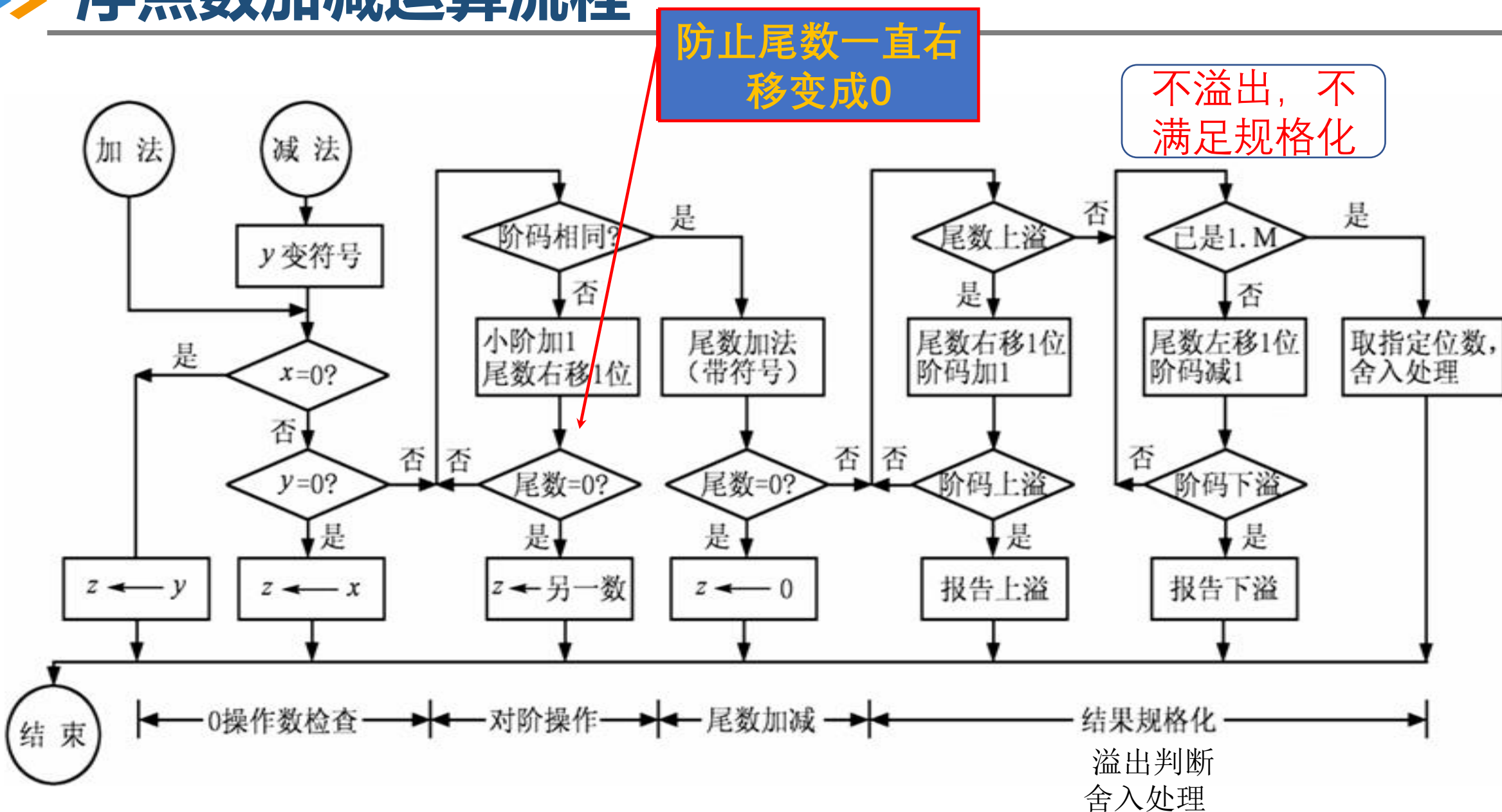
□ 下溢(结果绝对值太小)——置机器零；

□ 正常——运算结束；

➤ 接上例，运算结果的阶码 $E_S = 011 + 1 = 0100$

□ 未溢出！

>>> 浮点数加减运算流程



>>> 浮点数加减相关情况判断举例（需要理解并会判断）

对于 $float$ 类型，因 $1 < E < 255$ ，所以 $-126 \leq e \leq 127$

例1：对阶操作使尾数为零 **X得尾数右移70次，其阶码才是24，所以.....**

$$x = 0.11011011 \times 2^{-50}, y = 0.10101100 \times 2^{24}, \text{ 计算 } x + y$$

例2：尾数相加产生溢出 **X和y尾数相加，符号位产生进位，符号位10**

$$x = 0.11011011 \times 2^4, y = 1.10101100 \times 2^4, \text{ 计算 } x + y$$

例3：规格化阶码上溢 **Y规格化，当前尾数左移，阶码加一，则上溢**

$$x = 0.11011011 \times 2^{127}, y = 1.10101100 \times 2^{127}, \text{ 计算 } x + y$$

例4：规格化阶码下溢，假设规格化之前的运算结果如下

$$z = 0.00011011011 \times 2^{-125}$$

规格化对应的E阶码最小值为-126

当前尾数左移，阶码减一，则最终阶码-129，下溢

>>> 设 $x = 2^{010} \times 0.11011011$, $y = 2^{100} \times (-0.10101100)$, 求 $x + y$ 。

➤ 设浮点数的阶码用双符号位，尾数用单符号位的补码表示；

□ $[X]_{\text{浮}} = 00\ 010, 0.11011011$

□ $[Y]_{\text{浮}} = 00\ 100, 1.01010100$

① 求阶差并对阶

○ $\Delta E = E_X - E_Y = [E_X]_{\text{补}} + [-E_Y]_{\text{补}}$
 $= 00\ 010 + 11\ 100 = 11\ 110$

$\Delta E = -2 < 0$
则 E_Y 为结果阶码，修改浮点数 X

○ 则浮点数 X ，应使 M_X 右移 2 位， E_X 加 2，
 $[X]_{\text{浮}} = 00\ 100, 0.00110110(11)$, $E_S = 00\ 100$

② 尾数求和：

□ $M_S = 1.10001010(11)$

$$\begin{array}{r} 0.00110110(11) \\ + 1.01010100 \\ \hline 1.10001010(11) \end{array}$$

>>> 设 $x = 2^{010} \times 0.11011011$, $y = 2^{100} \times (-0.10101100)$, 求 $x + y$ 。

③ 和的规格化处理

$$M_S = 11.10001010(11)$$

□ 结果尾数的符号位与最高数值位相同, 应左规1位

□ 则 $M_S = 1.00010101(10)$, $E_S = 00\ 100 - 1 = 00\ 011$ 。

④ 舍入处理

□ 若采用0舍1入法

$$M_S = 1.00010110$$

$$\begin{array}{r} 1.00010101 \\ + \qquad \qquad \qquad 1 \\ \hline 1.00010110 \end{array}$$

⑤ 结果溢出判断

□ 阶码符号位为00, 和不溢出

$$\begin{aligned} \text{最终结果 } S &= X + Y = 00\ 011, 1.00010110 \\ &= 2^{011} \times (-0.11101010) \end{aligned}$$

>>> 课堂练习

已知: $x = -0.1000101 \times 2^{-111}$

$y = +0.0001010 \times 2^{-100}$

①用补码运算求 $x + y = ?$ 并判断是否溢出?

②用补码运算求 $x - y = ?$ 并判断是否溢出?

【解】

假设两数均以补码表示, 且采用双符号位, 先将它们变为规格化浮点数, 则它们的浮点表示分别为

$[x]_{\text{浮}} = 11\ 001, \ 11.0111011$

$[y]_{\text{浮}} = 11\ 100, \ 00.0001010 = 11\ 001, \ 00.1010000$

➤➤➤ 已知 $[x]_{\text{浮}} = 11\ 001, 11.0111011$, $[y]_{\text{浮}} = 11\ 001, 00.1010000$ 求 $x + y$ 。

<1> 求阶差并对阶

$E_x = E_y$, 即 ΔE 为0, 阶码相等, 无需对阶。

<2> 尾数求和

$M_s = 00.0\ 0\ 0\ 1\ 0\ 1\ 1$

$$\begin{array}{r} 11.0\ 1\ 1\ 1\ 0\ 1\ 1 \\ +\ 00.1\ 0\ 1\ 0\ 0\ 0\ 0 \\ \hline 00.0\ 0\ 0\ 1\ 0\ 1\ 1 \end{array}$$

<3>规格化处理

尾数运算结果的符号位与最高数值位同值, 应执行左规处理(阶码减1);

$M_s = 00.1011000$, $E_s = 10\ 110$

<4>舍入处理: 没有丢失位, 不必进行舍入。

<5>判溢出: 阶码符号位为10, 发生下溢, 故需置机器0。

>>> 已知 $[x]_{\text{浮}} = 11\ 001, 11.0111011$, $[y]_{\text{浮}} = 11\ 001, 00.1010000$ 求 $x - y$ 。

<1> 求阶差并对阶: $E_x = E_y$, 即 ΔE 为0, 阶码相等, 无需对阶。

<2> 尾数求差

$$[-M_y]_{\text{补}} = 11.0110000, \quad M_s = 10.1101011 \quad + \quad \begin{array}{r} 11.0111011 \\ 11.0110000 \\ \hline 10.1101011 \end{array}$$

<3>规格化处理: 两位符号位不同, 应执行右规1位的处理。

$$M_s = 11.0110101 \ (1), \quad E_s = 11\ 010.$$

<4>舍入处理 **0舍1入法**

丢失位高位为1, 故应进位, $M_s = 11.0110110$ 。

<5>判溢出

阶码符号位为11, 无溢出, 故结果为:

$$x - y = -0.1001010 \times 2^{-6}.$$

2009年考研统考题目 第13题

设浮点数的阶码和尾数均采用补码表示，且位数分别为5位和7位（均含2位符号位）。若有两个数 $X=2^7 \times 29/32$ ， $Y=2^5 \times 5/8$ ，则有浮点加法计算 $X+Y$ 最终结果是（ ）。

☐ A 00111 1100010

☐ B 00111 0100010

☐ C 01000 0010001

☒ D 发生溢出

$$X=1\ 1101 \times 2^{+2}$$

$$Y=101 \times 2^{+2}$$

提交

>>>2009年考研统考题目 第13题

设浮点数的阶码和尾数均采用补码表示，且位数分别为5位和7位（均含2位符号位）。若有两个数 $X=2^7 \times 29/32$ ， $Y=2^5 \times 5/8$ ，则有浮点加法计算 $X+Y$ 最终结果是（ ）。

答案解析

X 的浮点数格式为 00,111;00,11101（分号前为阶码，分号后为尾数），Y 的浮点数格式为 00,101;00,10100。然后根据浮点数的加法步骤进行运算。

第一步：对阶。X、Y 阶码相减，即 $00,111 - 00,101 = 00,111 + 11,0111 = 00,010$ ，可知 X 的阶码比 Y 的阶码大 2（这一步可直接目测）。根据小阶向大阶看齐的原则，将 Y 的阶码加 2，尾数右移 2 位，将 Y 变为 00,111;00,00101。

第二步：尾数相加。即 $00,11101 + 00,00101 = 01,00010$ ，尾数相加结果符号位为 01，故需右规。

第三步：规格化。将尾数右移 1 位，阶码加 1，得 $X+Y$ 为 01,000;00,10001。

第四步：判溢出。阶码符号位为 01，说明发生溢出。

2015年考研题 第14题

下列有关浮点数加减运算的叙述中，正确的是___。

- I. 对阶操作不会引起阶码上溢或下溢
- II. 右规和尾数舍入都可能引起阶码上溢
- III. 左规时可能引起阶码下溢
- IV. 尾数溢出时结果不一定溢出

A 仅 II、III

C 仅 I、III、IV

B 仅 I、II、IV

D I、II、III、IV

提交

2015年考研题 第14题

下列有关浮点数加减运算的叙述中，正确的是___。

- I. 对阶操作不会引起阶码上溢或下溢
- II. 右规和尾数舍入都可能引起阶码上溢
- III. 左规时可能引起阶码下溢
- IV. 尾数溢出时结果不一定溢出

A 仅 II、III

C 仅 I、III、IV

B 仅 I、II、IV

D I、II、III、IV

- 舍入误差：在舍入过程中，尾数的变化可能导致阶码的调整
- 例如，尾数右移时，如果移掉的位是1，尾数末尾加1，若采用0舍1入法，这可能导致尾数溢出，从而引起阶码上溢

提交

>>> 本章小结

- 掌握数据的不同表示方法，并能在不同的表示形式间转换；
 - 真值和机器数；原补移码、定点数等表示
 - 浮点数的表示、取值范围、规格化、IEEE754标准
- 理解数据校验的原理和方法
 - 奇偶校验
- 掌握定点数的运算，了解相关运算器的构成
 - 补码加减法运算、溢出的检测 and 判断方法掌握浮点数的加减法运算方法和步骤